SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Integrated Experiments

Weightage: 40%

QUESTIONS: **Total Marks: 30**

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Experiment 1: Responsive Layout Design

Suppose that a retail website is not responsive across mobile and tablet devices, causing misalignment of content and improper spacing.
(a) Analyze the possible reasons for lack of responsiveness in the existing layout.
(b) Design a responsive webpage layout using **CSS Flexbox or Grid**, ensuring proper alignment of header, navigation, content, and footer.
(c) Demonstrate how the layout adapts to **different screen sizes (mobile, tablet, desktop)** using media queries.
(d) Implement spacing, alignment, and distribution techniques to improve visual consistency.
(e) Evaluate the effectiveness of the responsive design and suggest further improvements.

Experiment 2: Form Validation using JavaScript

Suppose that a registration form accepts invalid email and phone number inputs, leading to incorrect data submission.
(a) Analyze the issues in the existing form validation mechanism.
(b) Design input fields for email and phone number with appropriate HTML attributes.
(c) Implement **JavaScript validation using Regular Expressions** for both email and phone number.
(d) Develop a mechanism to display **real-time error messages** for invalid inputs.
(e) Evaluate the validation system and suggest enhancements for better user experience and security.

Experiment 3: CSS Layout Debugging

Suppose that a webpage layout overlaps when the browser window is resized, affecting usability.
(a) Analyze the CSS properties responsible for layout overlapping issues.
(b) Examine the role of the **CSS box model (margin, border, padding, content)** in layout design.
(c) Debug the layout using appropriate **positioning techniques (relative, absolute, flex, grid)**.
(d) Modify the CSS to ensure proper alignment and prevent overlapping during resizing.
(e) Evaluate the corrected layout and suggest best practices for maintaining consistency.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools

		with proper configuration			
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

DETAILED SOLUTIONS

Assessment Tool 1 · CO-2 · Internet Programming

The original three-page assessment paper is reproduced first without changing its content. The detailed solutions begin on the following page and address each sub-question with complete HTML/CSS/JavaScript code and illustrative browser-view screenshots.

Experiment 1: Responsive Layout Design

Aim

To design a retail webpage that maintains clear alignment and readable spacing on mobile, tablet and desktop screens using CSS Grid, Flexbox and media queries.

(a) Analysis of possible reasons for poor responsiveness

- Fixed pixel widths and heights prevent elements from shrinking naturally on narrow screens.
- Navigation links and product cards may remain in one row, causing horizontal overflow.
- Absolute positioning and hard-coded offsets can make elements overlap when text wraps or viewport dimensions change.
- Missing viewport metadata can cause mobile browsers to render the page at a wider virtual viewport.
- Inconsistent margins, padding and box sizing can create unexpected widths and uneven spacing.
- Images or long text without max-width and wrapping rules may exceed their parent containers.

(b)–(d) Design, implementation and spacing

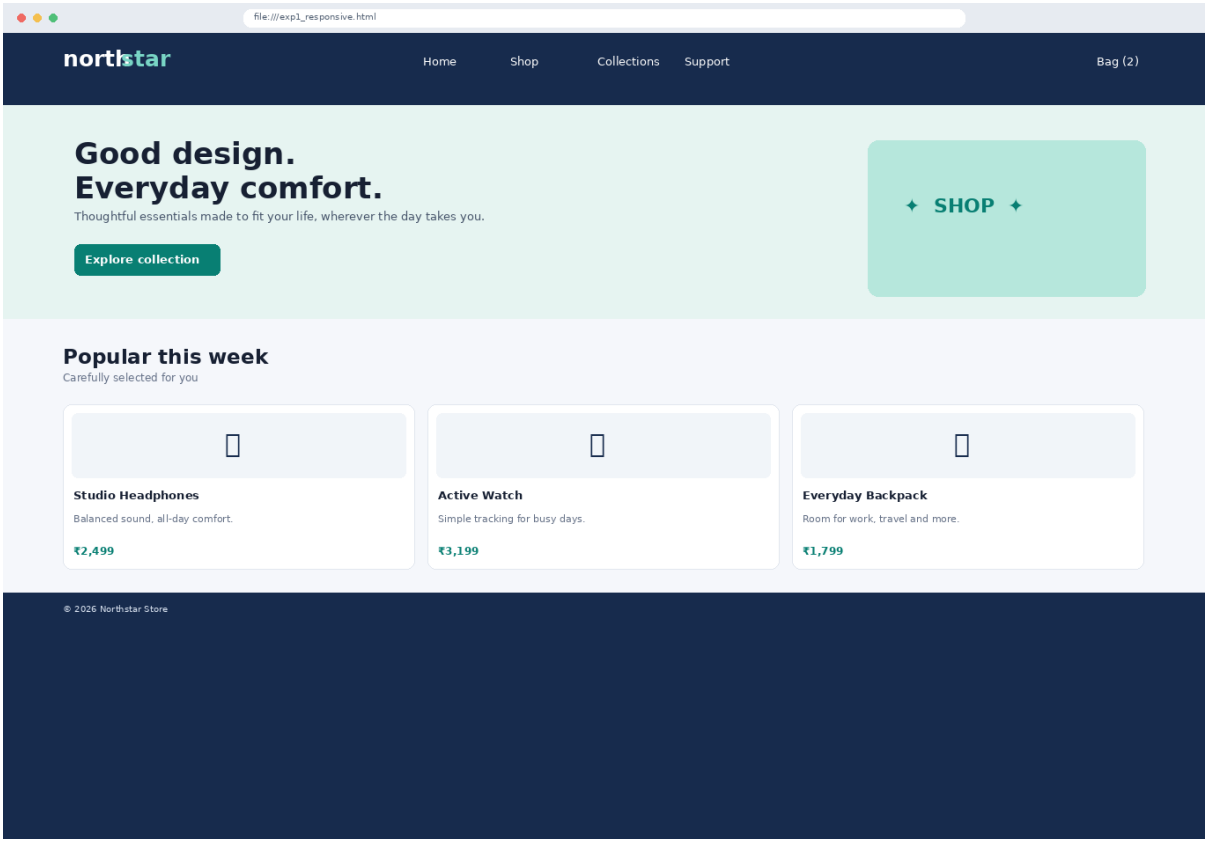
The example uses a flexible header and navigation, a two-column hero, and a three-column product grid on desktop. At widths of 800 px or less, the hero becomes one column and products become two columns. At widths of 520 px or less, the product grid becomes one column and the header stacks vertically. The universal box-sizing rule makes dimensions predictable; minmax(0, 1fr), flexible gaps and percentage-based padding help prevent overflow.

Complete code (save as exp1_responsive.html)

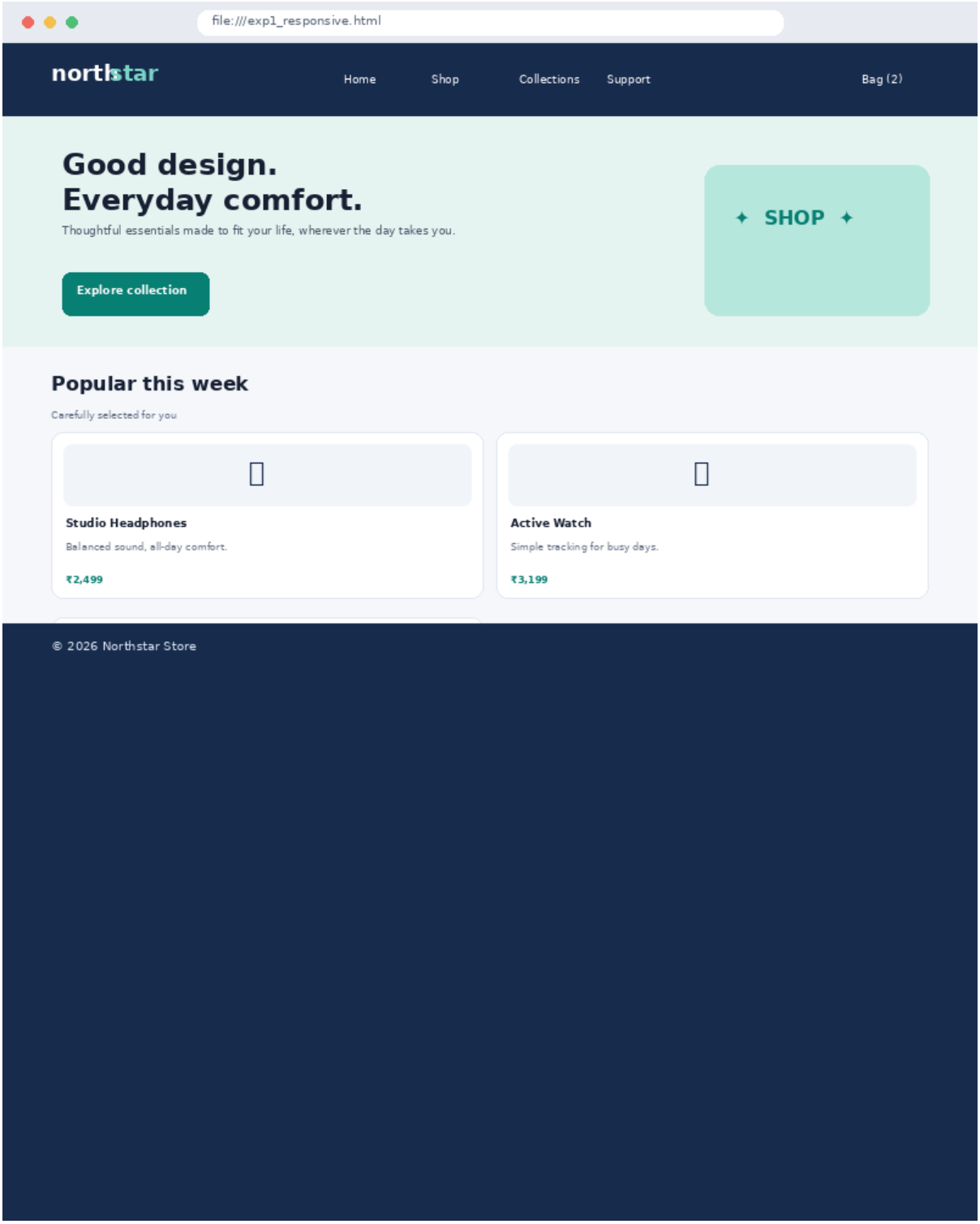
```
<!doctype html>
<html lang="en"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1">
<title>Northstar — Everyday essentials</title>
<style>
*{box-sizing:border-box}body{margin:0;font:16px/1.5 Arial,sans-serif;color:#172033;background:#f5f7fb}
header{background:#172b4d;color:#fff;padding:18px 5vw;display:flex;align-items:center;justify-content:space-between;gap:18px;flex-wrap:wrap}
.brand{font-size:1.35rem;font-weight:700;letter-spacing:.4px}.brand span{color:#79d5c5}
nav{display:flex;gap:20px;flex-wrap:wrap}nav a{color:#fff;text-decoration:none;font-size:.95rem}
.hero{padding:36px 5vw;background:#e6f4f1;display:grid;grid-template-columns:1.5fr 1fr;gap:24px;align-items:center}
.hero h1{font-size:clamp(1.8rem,4vw,3rem);line-height:1.1;margin:0 0 12px}.hero p{margin:0 0 18px;color:#46546a}
.cta{display:inline-block;background:#087f73;color:#fff;text-decoration:none;padding:11px 17px;border-radius:7px;font-weight:700}
.hero-art{min-height:160px;border-radius:16px;background:linear-gradient(135deg,#b6e7dc,#f8d8ae);display:flex;align-items:center;justify-
content:center;font-size:4.2rem}
main{padding:30px 5vw}.section-head{display:flex;justify-content:space-between;align-items:end;gap:12px;margin-bottom:18px;flex-wrap:wrap}.section-
head h2{margin:0;font-size:1.5rem}.section-head p{margin:0;color:#647087}
.products{display:grid;grid-template-columns:repeat(3,minmax(0,1fr));gap:18px}
.card{background:white;border:1px solid #e2e7ef;border-radius:12px;padding:18px;min-width:0;box-shadow:0 4px 14px #1c2b4610}
.product-icon{height:90px;background:#f1f5f9;border-radius:9px;display:flex;align-items:center;justify-content:center;font-size:2.7rem;margin-
bottom:12px}
.card h3{margin:0 0 6px;font-size:1.05rem}.card p{margin:0 0 12px;color:#657187;font-size:.92rem}.price{font-weight:700;color:#087f73}
footer{padding:22px 5vw;background:#172b4d;color:#dce5f2;display:flex;justify-content:space-between;gap:12px;flex-wrap:wrap;font-size:.9rem}
@media(max-width:800px){.hero{grid-template-columns:1fr}.hero-art{min-height:110px}.products{grid-template-columns:repeat(2,minmax(0,1fr))}}
@media(max-width:520px){header{align-items:flex-start;flex-direction:column}nav{gap:12px}.hero{padding-top:26px;padding-
bottom:26px}.products{grid-template-columns:1fr}main{padding-top:22px}.card{display:grid;grid-template-columns:78px 1fr;column-gap:13px}.product-
icon{height:78px;grid-row:span 3;margin:0}.card h3{align-self:end}}
</style></head><body>
<header><div class="brand">north<span>star</span></div><nav><a href="#">Home</a><a href="#">Shop</a><a href="#">Collections</a><a
href="#">Support</a></nav><div>Bag (2)</div></header>
<section class="hero"><div><h1>Good design.<br>Everyday comfort.</h1><p>Thoughtful essentials made to fit your life, wherever the day takes you.</p><a
class="cta" href="#products">Explore collection</a></div><div class="hero-art" aria-label="Decorative shopping illustration">✦ 🛍️ ✦</div></section>
<main id="products"><div class="section-head"><h2>Popular this week</h2><p>Carefully selected for you</p></div><div class="products">
<article class="card"><div class="product-icon">🎧</div><h3>Studio Headphones</h3><p>Balanced sound, all-day comfort.</p><div
class="price">₹2,499</div></article>
<article class="card"><div class="product-icon">⌚</div><h3>Active Watch</h3><p>Simple tracking for busy days.</p><div
```

```
class="price">₹3,199</div></article>
<article class="card"><div class="product-icon">🎒</div><h3>Everyday Backpack</h3><p>Room for work, travel and more.</p><div
class="price">₹1,799</div></article>
</div></main><footer><span>© 2026 Northstar Store</span><span>Help · Delivery · Returns</span></footer></body></html>
```

Browser-view screenshots and responsive evidence



Desktop viewport (1365 × 950): three product cards across; hero and navigation align in a wide layout.



Tablet viewport (800 × 1000): hero content stacks and product cards use two columns.

file:///exp1_responsive.html

northstar

Home Shop Collections

Good design. Everyday comfort.

Thoughtful essentials made to fit your life, wherever the day takes you.

Explore collection

✦ SHOP ✦

Popular this week

Carefully selected for you



Studio Headphones

Balanced sound, all-day comfort.

₹2,499



Active Watch

Simple tracking for busy days.

₹3,199

© 2026 Northstar Store

Mobile viewport (390 × 1000): navigation wraps, product cards stack and content remains within the screen.

(e) Evaluation and further improvements

The design uses content-driven sizing and breakpoints to preserve readability and avoid horizontal scrolling at the tested widths. The layout is more robust than a fixed-width design because columns can shrink and stack as space decreases. Further improvements include testing intermediate widths and 200% zoom, using real product images with descriptive alt text, adding visible keyboard focus states, checking contrast, and validating the page on multiple browsers and devices.

Experiment 2: Form Validation using JavaScript

Aim

To validate email addresses and mobile numbers on the client side, display immediate and accessible error messages, and prevent invalid form submission.

(a) Analysis of existing validation issues

- The form may only check whether fields are empty and may not validate their formats.
- Inputs may have no required, type, inputmode or length attributes to guide users.
- Validation only after submission delays feedback and makes correction harder.
- Error messages may be generic, not associated with the relevant field, or not announced to assistive technology.
- Client-side validation alone is not a security boundary; the server must validate data again.

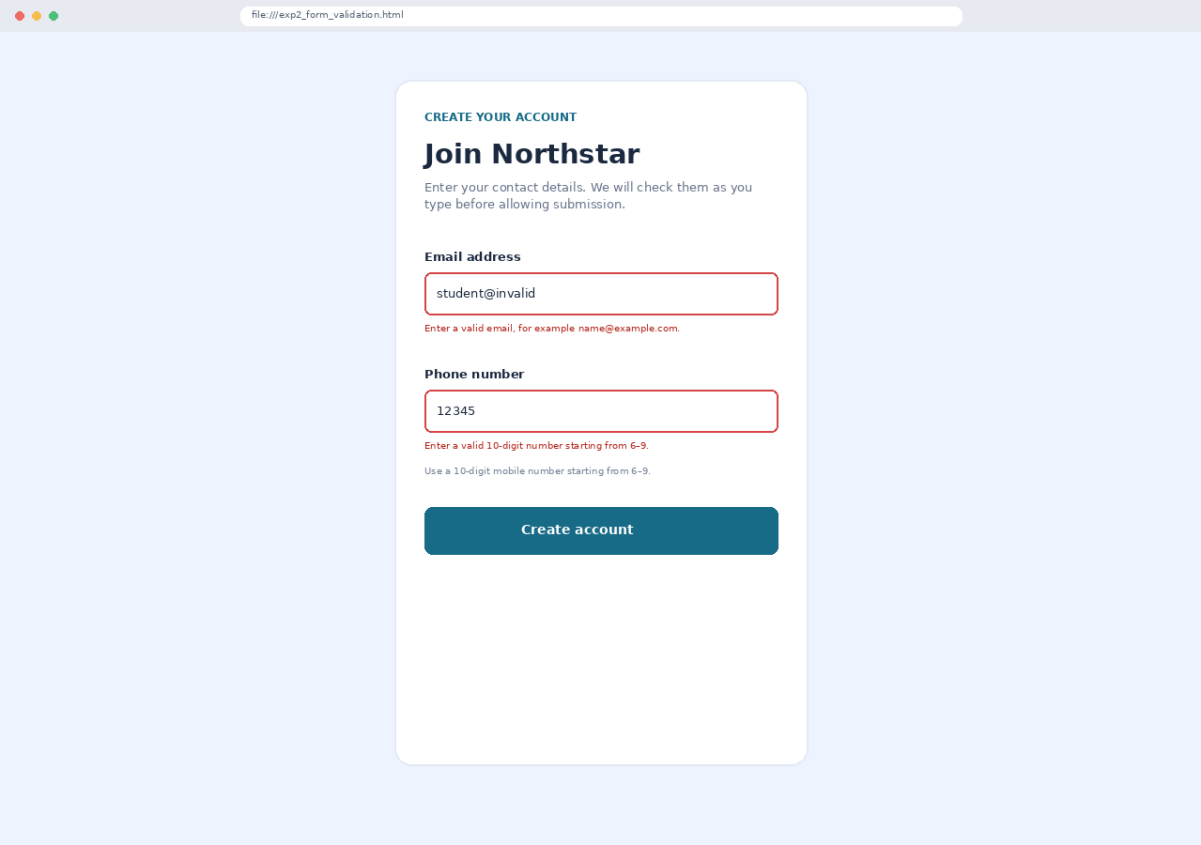
(b)–(d) Input design, regular expressions and real-time feedback

The email field uses type="email" and required. The phone field uses type="tel", inputmode="numeric", required and maxlength="10". JavaScript validates email with a regular expression and phone numbers with /^[6-9]\d{9}$/. Input listeners update error messages as the user types. The submit handler prevents submission when either field is invalid and shows a success message only when both checks pass.

Complete code (save as exp2_form_validation.html)

```
<!doctype html>
<html lang="en"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1"><title>Account registration</title>
<style>*{box-sizing:border-box}body{margin:0;min-height:100vh;display:grid;place-items:center;background:linear-
gradient(135deg,#eef4ff,#f8fbff);font:16px Arial,sans-
serif;color:#1b2940;padding:22px}.panel{width:min(100%,470px);background:#fff;padding:32px;border:1px solid #e3eaf5;border-radius:18px;box-
shadow:0 18px 55px #243b6417}.eyebrow{color:#176b87;font-size:.78rem;font-weight:700;letter-spacing:1.5px;text-transform:uppercase}h1{margin:8px
0;font-size:1.8rem} .intro{color:#65738a;line-height:1.5;margin:0 0 24px}.field{margin:17px 0}label{display:block;font-weight:700;font-size:.92rem;margin-
bottom:7px}input{width:100%;padding:12px 13px;border:1px solid #cbd5e1;border-radius:8px;font:inherit;outline:none}input:focus{border-
color:#2476cf;box-shadow:0 0 0 3px #2476cf20}.error{color:#b42318;font-size:.82rem;min-height:18px;margin-top:5px}.success{display:none;margin-
top:15px;padding:12px;border-radius:8px;background:#e8f7ee;color:#17643b}.submit{width:100%;border:0;border-
radius:8px;padding:13px;background:#176b87;color:white;font-weight:700;font-size:1rem;cursor:pointer}.hint{color:#718096;font-size:.8rem;margin-
top:5px}</style></head><body>
<section class="panel"><div class="eyebrow">Create your account</div><h1>Join Northstar</h1><p class="intro">Enter your contact details. We will check
them as you type before allowing submission.</p>
<form id="registerForm" novalidate>
<div class="field"><label for="email">Email address</label><input id="email" name="email" type="email" autocomplete="email" required
placeholder="you@example.com" aria-describedby="emailError"><div class="error" id="emailError" aria-live="polite"></div></div>
<div class="field"><label for="phone">Phone number</label><input id="phone" name="phone" type="tel" inputmode="numeric" autocomplete="tel"
required maxlength="10" placeholder="10-digit mobile number" aria-describedby="phoneError"><div class="error" id="phoneError"
aria-live="polite"></div><div class="hint">Use a 10-digit mobile number starting from 6–9.</div></div>
<button class="submit" type="submit">Create account</button><div class="success" id="success" role="status">Details validated successfully. Demo
submission complete.</div></form></section>
<script>
const form=document.getElementById('registerForm'), email=document.getElementById('email'), phone=document.getElementById('phone');
const emailError=document.getElementById('emailError'), phoneError=document.getElementById('phoneError'),
success=document.getElementById('success');
const emailPattern=/^[^\s@]+@[^\s@]+\.[A-Za-z]{2,}$/;
const phonePattern=/^[6-9]\d{9}$/;
function validateEmail(){const v=email.value.trim();let msg='';if(!v)msg='Email address is required.';else if(!emailPattern.test(v))msg='Enter a valid email,
for example name@example.com.';emailError.textContent=msg;email.setAttribute('aria-invalid',String(Boolean(msg)));return !msg}
function validatePhone(){const v=phone.value.trim();let msg='';if(!v)msg='Phone number is required.';else if(!phonePattern.test(v))msg='Enter a valid 10-
digit number starting from 6–9.';phoneError.textContent=msg;phone.setAttribute('aria-invalid',String(Boolean(msg)));return !msg}
email.addEventListener('input',()=>{validateEmail();success.style.display='none'});phone.addEventListener('input',()=>{phone.value=phone.value.replace(/
\D/g,'').slice(0,10);validatePhone();success.style.display='none'});
form.addEventListener('submit',e=>{e.preventDefault();const
validEmail=validateEmail(),validPhone=validatePhone();if(validEmail&&validPhone)success.style.display='block';else success.style.display='none'});
</script></body></html>
```

Browser-view screenshot



Illustrative form state with invalid email and phone entries. In the live HTML file, messages update as the user types; valid demo inputs show a confirmation message.

Suggested test cases

Input	Expected result	Reason
Email: empty	Error shown	Required field
Email: user@site	Error shown	Missing valid domain suffix
Email: student@example.com	Accepted by demo regex	Expected email pattern
Phone: 12345	Error shown	Not 10 digits / invalid prefix
Phone: 9876543210	Accepted by demo regex	10 digits, starts 6–9
Phone: letters/symbols	Non-digits removed while typing	Input is normalized

(e) Evaluation, user experience and security

Immediate feedback helps users fix mistakes before submission. Labels, aria-describedby, aria-live and aria-invalid improve accessibility. The regular expressions are suitable for this assignment’s simple format checks but do not prove that an email address exists or that a phone number belongs to the user. For production, repeat validation on the server, normalize inputs, use HTTPS, rate-limit sensitive endpoints, and verify contact ownership with a confirmation link or one-time code. Avoid displaying sensitive details in error messages or logs.

Experiment 3: CSS Layout Debugging

Aim

To diagnose overlapping elements and implement a stable responsive layout using the CSS box model, Grid, Flexbox and appropriate positioning.

(a) CSS properties that can cause overlapping

- Absolute or fixed positioning removes elements from normal document flow and can cause them to cover other content.
- Fixed widths combined with large padding, borders or margins can make an element wider than its parent.
- Negative margins, excessive z-index values and hard-coded top/left offsets can create unintended overlap.
- Non-wrapping navigation, long unbroken strings and inflexible grid columns can overflow narrow screens.
- Fixed heights may clip text when font size increases or content wraps.

(b) Role of the CSS box model

Each element consists of content, padding, border and margin. With the default content-box model, the declared width applies only to the content, so padding and borders increase the rendered width. Applying box-sizing: border-box includes padding and borders within the declared width, making responsive calculations easier. Padding creates internal space, border outlines the box, margin separates adjacent boxes, and content holds the actual text or media.

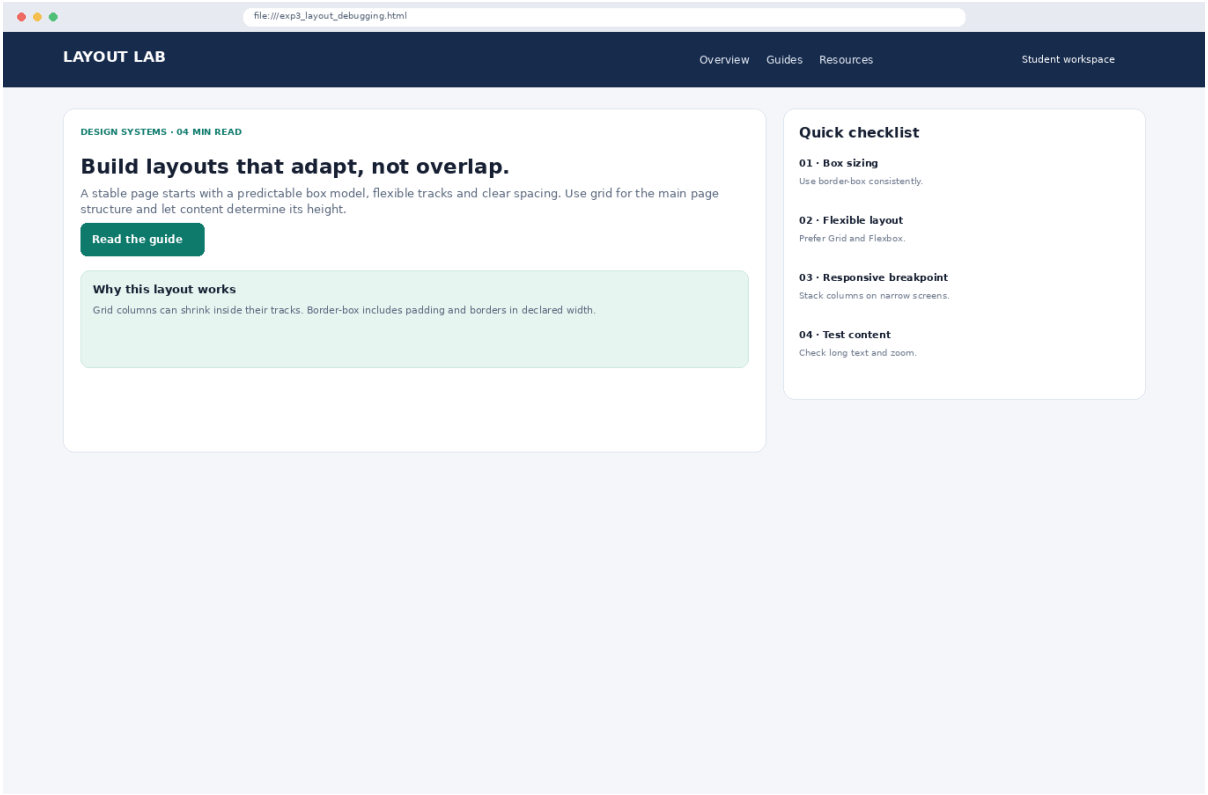
(c)–(d) Debugging strategy

- Use display: grid with minmax(0, 2fr) and minmax(230px, 1fr) for a flexible main/sidebar layout.
- Use a max-width container and auto margins to center content without stretching it excessively.
- Apply box-sizing: border-box globally and min-width: 0 to grid children to prevent intrinsic content from forcing overflow.
- At 720 px or less, change the layout to one column and reduce padding; allow navigation and footer content to wrap.
- Reserve absolute positioning for decorative or intentionally overlaid elements, not the main page structure.

Complete code (save as exp3_layout_debugging.html)

```
<!doctype html>
<html lang="en"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1"><title>Layout Lab</title><style>
*{box-sizing:border-box}body{margin:0;background:#f4f6fa;color:#172033;font:16px/1.5 Arial,sans-serif}.topbar{background:#172b4d;color:white;padding:16px max(20px,5vw);display:flex;align-items:center;justify-content:space-between;gap:15px;flex-wrap:wrap}.logo{font-weight:800;font-size:1.2rem}.topbar nav{display:flex;gap:18px;flex-wrap:wrap}.topbar a{color:#e9eff8;text-decoration:none}.layout{max-width:1120px;margin:28px auto;padding:0 22px;display:grid;grid-template-columns:minmax(0,2fr) minmax(230px,1fr);gap:22px;align-items:start}.content,.sidebar,.feature{background:white;border:1px solid #e0e6ef;border-radius:13px;padding:22px;min-width:0}.eyebrow{font-size:.78rem;color:#0d7a6c;font-weight:700;letter-spacing:1px;text-transform:uppercase}.content h1{font-size:clamp(1.6rem,3vw,2.3rem);line-height:1.15;margin:8px 0 12px}.content p{color:#59677d}.feature{background:#e7f5f1;border-color:#cde8e0;margin-top:18px}.feature h2{margin:0 0 7px;font-size:1.1rem}.sidebar h2{font-size:1.1rem;margin:0 0 12px}.task{padding:12px 0;border-bottom:1px solid #edf0f5}.task:last-child{border:0}.task strong{display:block}.task span{color:#6b778b;font-size:.9rem}.action{display:inline-block;background:#0d7a6c;color:white;text-decoration:none;padding:10px 14px;border-radius:7px;font-weight:700;margin-top:6px}.footer{max-width:1120px;margin:0 auto;padding:18px 22px 30px;color:#68758a;font-size:.9rem;display:flex;justify-content:space-between;gap:12px;flex-wrap:wrap}
@media(max-width:720px){.layout{grid-template-columns:1fr;margin-top:18px}.topbar{align-items:flex-start;flex-direction:column}.sidebar{order:2}.content,.sidebar{padding:18px}}
</style></head><body><header class="topbar"><div class="logo">LAYOUT LAB</div><nav><a href="#">Overview</a><a href="#">Guides</a><a href="#">Resources</a></nav><div>Student workspace</div></header>
<div class="layout"><main class="content"><div class="eyebrow">Design systems · 04 min read</div><h1>Build layouts that adapt, not overlap.</h1><p>A stable page starts with a predictable box model, flexible tracks and clear spacing. Use grid for the main page structure and let content determine its height.</p><a class="action" href="#">Read the guide</a><section class="feature"><h2>Why this layout works</h2><p>Grid columns use minmax(0, ...) so long content can shrink inside its track. The global border-box rule includes padding and borders in each element’s declared width.</p></section></main>
<aside class="sidebar"><h2>Quick checklist</h2><div class="task"><strong>01 · Box sizing</strong><span>Use border-box consistently.</span></div><div class="task"><strong>02 · Flexible layout</strong><span>Prefer Grid and Flexbox.</span></div><div class="task"><strong>03 · Responsive breakpoint</strong><span>Stack columns on narrow screens.</span></div><div class="task"><strong>04 · Test content</strong><span>Check long text and zoom.</span></div></aside></div><footer class="footer"><span>© 2026 Layout Lab</span><span>Accessible · Responsive · Maintainable</span></footer></body></html>
```

Browser-view screenshots and verification



Desktop: the content panel and checklist occupy separate grid columns with consistent spacing.

file:///exp3_layout_debugging.html

LAYOUT LAB

Overview Guides Resources

DESIGN SYSTEMS · 04 MIN READ

Build layouts that adapt, not overlap.

A stable page starts with a predictable box model, flexible tracks and clear spacing. Use grid for the main page structure and let content determine its height.

Read the guide

Why this layout works

Grid columns can shrink inside their tracks. Border-box includes padding and borders in declared width.

Quick checklist

01 · Box sizing

Use border-box consistently.

02 · Flexible layout

Prefer Grid and Flexbox.

Mobile: the main content and checklist stack vertically without covering each other.

(e) Evaluation and best practices

The corrected layout keeps content in normal document flow, uses flexible grid tracks, and switches to a single column on narrow screens. These choices reduce overlap and improve maintainability. Before submission, test at several viewport widths, inspect browser developer tools for overflow, verify keyboard navigation, test long headings and enlarged text, and avoid fixed heights for text-heavy components. The screenshots are illustrative browser-view mockups of the specified viewport layouts; the supplied HTML files should be opened in a browser for live execution.

Conclusion

The three experiments demonstrate responsive page composition, client-side form validation, and systematic CSS layout debugging. CSS Grid and Flexbox provide flexible structure, media queries adapt the interface to different viewports, and JavaScript regular expressions provide immediate format checks. Accessibility and server-side validation remain essential for a production-quality implementation.